

Avaliação Prática de Linguagem C

Instruções Gerais

- Utilize exclusivamente a biblioteca `<stdio.h>`.
- Duração: 120 minutos (2 horas).
- Os números à esquerda dos códigos indicam as linhas e não fazem parte do programa.

Tabela de Pontuação

Seção	Descrição	Quantidade	Pontuação Unitária	Pontuação Máxima
Parte I	Identificação de Erros (Fechadas)	10 Questões (alternativas a–f)	0,50 pto por questão	5,00 pto
Parte II	Escrita de Código (Abertas)	5 Questões	1,00 pto por questão	5,00 pto
Total				10,00 pto

Parte I: Identificação de Erros

Em cada questão a seguir, compare o código com a descrição e a saída esperada e marque a **única** alternativa que indica **onde** está o erro e **por que** ele é um erro. O erro pode impedir a compilação ou fazer o programa não gerar a saída esperada. Se o código estiver correto, marque a alternativa f.

Questão 1: Média da Turma

Descrição: Lê a quantidade de alunos e a nota inteira de cada um e exibe a média da turma.

Entrada: 4 → 7 8 6 9

Saída Esperada: Média: 7.500000

```
1 #include <stdio.h>
2
3 float media(int v[], int n) {
4     float soma = 0;
5     for (int i = 0; i < n; i++) {
6         soma += v[i];
7     }
8     return soma / n;
9 }
10
11 int main() {
12     int n;
13     int notas[50];
14     scanf("%d", &n);
15     for (int i = 0; i < n; i++) {
16         scanf("%d", notas[i]);
17     }
18     printf("Média: %f\n", media(notas, n));
19     return 0;
20 }
```

- a) Linha 3 — o parâmetro deveria ser `int v[50]`, pois a função precisa saber o tamanho exato do vetor.
- b) Linha 8 — dividir um `float` por um `int` não é permitido; `n` teria que ser declarado como `float`.
- c) Linha 18 — o vetor deveria ser enviado como `¬as`, pois a função precisa receber o endereço dele.
- d) Linha 16 — falta o `&` em `notas[i]`; cada posição do vetor é um `int` comum e o `scanf` exige o endereço.
- e) Linha 4 — `soma` deveria ser `int`, porque ela recebe apenas valores inteiros vindos do vetor de notas.
- f) O código não tem nenhum erro.

Questão 2: Calculadora de Opções

Descrição: Lê uma opção: com 1 exibe apenas a soma de 10 e 4; com 2, apenas a subtração.

Entrada: 1

Saída Esperada: Soma: 14

```
1 #include <stdio.h>
2
3 int main() {
4     int opcao;
5     int a = 10;
6     int b = 4;
7     scanf("%d", &opcao);
8     switch (opcao) {
9         case 1:
10             printf("Soma: %d\n", a + b);
11         case 2:
12             printf("Subtracao: %d\n", a - b);
13             break;
14         default:
15             printf("Opcao invalida\n");
16     }
17     return 0;
18 }
```

- a) Linha 8 — o switch só aceita variáveis do tipo char, então opcao deveria ser declarada como char.
- b) Linhas 10-11 — falta o break ao fim do case 1; ao escolher 1 o programa também executa o case 2.
- c) Linha 14 — o default precisa terminar com um break, senão o switch inteiro não compila.
- d) Linha 7 — o scanf deveria ficar dentro do switch, para ler a opção antes de cada um dos case.
- e) Linha 12 — a conta $a - b$ não pode ficar dentro do printf; o resultado precisa estar numa variável.
- f) O código não tem nenhum erro.

Questão 3: Troca de Valores

Descrição: Usa uma função para trocar os valores das variáveis x e y.

Saída Esperada: x = 7, y = 3

```
1 #include <stdio.h>
2
3 void troca(int a, int b) {
4     int temp = a;
5     a = b;
6     b = temp;
7 }
8
9 int main() {
10     int x = 3;
11     int y = 7;
12     troca(x, y);
13     printf("x = %d, y = %d\n", x, y);
14     return 0;
15 }
```

- a) Linha 4 — a variável temp não pode ser criada dentro de uma função cujo tipo de retorno é void.
- b) Linha 12 — uma função void não pode ser chamada sozinha; o resultado dela precisa ir para uma variável.
- c) Linha 6 — as atribuições estão fora de ordem; $b = temp$ deveria vir antes de $a = b$ para a troca funcionar.
- d) Linha 13 — o printf não consegue imprimir duas variáveis int diferentes dentro do mesmo comando.
- e) Linha 3 — a troca acontece só nas cópias locais; seria preciso usar `int *a, int *b` e chamar `troca(&x, &y)`.
- f) O código não tem nenhum erro.

Questão 4: Maior Valor do Vetor

Descrição: Lê n números para um vetor e exibe o maior deles, mesmo que todos sejam negativos.

Entrada: 3 → -5 -1 -8

Saída Esperada: Maior: -1

```
1 #include <stdio.h>
2
3 int maior(int v[], int n) {
4     int m = v[0];
5     for (int i = 1; i < n; i++) {
6         if (v[i] > m) {
7             m = v[i];
8         }
9     }
10    return m;
11 }
12
13 int main() {
14     int n;
15     int valores[20];
16     scanf("%d", &n);
17     for (int i = 0; i < n; i++) {
18         scanf("%d", &valores[i]);
19     }
20     printf("Maior: %d\n", maior(valores, n));
21     return 0;
22 }
```

- a) Linha 5 — o for deveria começar em $i = 0$, senão a primeira posição $v[0]$ nunca é comparada.
- b) Linha 18 — o & em $\&valores[i]$ está sobrando, pois o vetor já é passado para o scanf como endereço.
- c) Linha 4 — m deveria começar valendo 0, pois $v[0]$ ainda não tem valor quando a função é chamada.
- d) Linha 10 — o return está fora do for, então a função sempre devolve apenas o primeiro valor do vetor.
- e) Linha 20 — não é possível chamar maior dentro do printf; o retorno precisa ir antes para uma variável.
- f) O código não tem nenhum erro.

Questão 5: Contagem da Letra a

Descrição: Lê uma palavra e usa uma função para contar quantas letras a ela possui.

Entrada: banana

Saída Esperada: Letras a: 3

```
1 #include <stdio.h>
2
3 int conta_a(char frase[]) {
4     int total = 0;
5     for (int i = 0; frase[i] != '\0'; i++) {
6         if (frase[i] == "a") {
7             total++;
8         }
9     }
10    return total;
11 }
12
13 int main() {
14     char palavra[50];
15     scanf("%s", palavra);
16     printf("Letras a: %d\n", conta_a(palavra));
17     return 0;
18 }
```

- a) Linha 6 — "a" com aspas duplas é uma string; para comparar um único char o correto seria usar 'a'.
- b) Linha 5 — a condição $frase[i] != '\0'$ nunca fica falsa, pois strings em C não têm um marcador de fim.
- c) Linha 15 — falta o & antes de palavra, pois o scanf precisa receber o endereço da variável lida.
- d) Linha 3 — uma função não pode receber uma string como parâmetro, só números e caracteres do tipo char.
- e) Linha 7 — o operador ++ só funciona na variável de controle do for, e não em contadores como total.
- f) O código não tem nenhum erro.

Questão 6: Soma de Vetor

Descrição: Usa uma função para somar todos os elementos de um vetor.

Saída Esperada: Total: 16

```
1 #include <stdio.h>
2
3 int soma_vetor(int v[], int n) {
4     int soma;
5     for (int i = 0; i < n; i++) {
6         soma = 0;
7         soma += v[i];
8     }
9     return soma;
10 }
11
12 int main() {
13     int valores[4] = {2, 5, 1, 8};
14     printf("Total: %d\n", soma_vetor(valores, 4));
15     return 0;
16 }
```

- a) Linha 4 — uma variável sem valor inicial não pode ser o return de uma função que tem o tipo int.
- b) Linha 13 — um vetor inicializado com {} não pode ser enviado como parâmetro para outra função.
- c) Linha 6 — soma = 0 está dentro do for e zera o acumulador a cada volta; deveria ficar antes do laço.
- d) Linha 5 — com i < n a última posição do vetor fica de fora da soma; o correto seria usar i <= n.
- e) Linha 7 — o operador += não funciona com posições de vetor, apenas com variáveis simples como soma.
- f) O código não tem nenhum erro.

Questão 7: Função de Fatorial

Descrição: Lê um número e calcula o seu fatorial (o produto de 1 até n).

Entrada: 5

Saída Esperada: Fatorial: 120

```
1 #include <stdio.h>
2
3 int fatorial(int n) {
4     int resultado = 1;
5     int i = 1;
6     while (i <= n) {
7         resultado = resultado * i;
8     }
9     return resultado;
10 }
11
12 int main() {
13     int numero;
14     scanf("%d", &numero);
15     printf("Fatorial: %d\n", fatorial(numero));
16     return 0;
17 }
```

- a) Linha 4 — resultado deveria começar em 0, porque toda variável acumuladora precisa iniciar em zero.
- b) Linha 6 — o while não aceita <= na condição; somente < e > podem controlar laços de repetição.
- c) Linha 15 — a função não pode ser chamada no printf, pois recebe uma variável que veio do scanf.
- d) Linha 9 — o return deveria ficar dentro do while, para devolver o valor a cada nova multiplicação.
- e) Linhas 6-8 — i nunca é incrementado dentro do while, a condição nunca fica falsa e o laço é infinito.
- f) O código não tem nenhum erro.

Questão 8: Validação de Nota

Descrição: Repete a leitura enquanto a nota digitada estiver fora do intervalo de 0 a 10.

Entrada: 15, depois 8

Saída Esperada: Nota registrada: 8

```
1 #include <stdio.h>
2
3 int main() {
4     int nota;
5     do {
6         scanf("%d", &nota);
7     } while (nota < 0 && nota > 10);
8     printf("Nota registrada: %d\n", nota);
9     return 0;
10 }
```

- a) Linha 7 — o ; depois do while encerra o laço antes da hora e deveria ser removido do código.
- b) Linha 7 — nenhum número é menor que 0 e maior que 10 ao mesmo tempo; com && nunca repete, o certo é ||.
- c) Linha 5 — o do precisa de uma condição entre parênteses, assim como acontece com o while e o if.
- d) Linha 6 — o scanf não pode ficar dentro de um laço, pois só consegue ler o teclado uma vez por programa.
- e) Linha 8 — nota só existe dentro do bloco do do-while, então não pode ser usada no printf depois.
- f) O código não tem nenhum erro.

Questão 9: Média de Duas Notas

Descrição: Usa uma função para calcular a média entre as notas 7.0 e 8.0.

Saída Esperada: Media: 7.500000

```
1 #include <stdio.h>
2
3 int media(float a, float b) {
4     float soma = a + b;
5     return soma / 2;
6 }
7
8 int main() {
9     float n1 = 7.0;
10    float n2 = 8.0;
11    float resultado = media(n1, n2);
12    printf("Media: %f\n", resultado);
13    return 0;
14 }
```

- a) Linha 5 — dividir um float por um inteiro como 2 não é permitido; o certo seria escrever soma / 2.0.
- b) Linha 12 — o especificador %f não serve para float; no printf o correto seria usar sempre %lf.
- c) Linha 11 — o retorno de uma função não pode ser guardado numa variável declarada nessa mesma linha.
- d) Linha 3 — a função devolve int, então o 7.5 perde a parte decimal e vira 7; o tipo deveria ser float.
- e) Linha 4 — soma precisaria ser declarada fora da função, senão não poderia ser usada pelo return.
- f) O código não tem nenhum erro.

Questão 10: Desconto em Vetor

Descrição: Aplica R\$ 30 de desconto em cada preço do vetor, sem deixar nenhum preço negativo.

Saída Esperada:

20

0

50

```
1  #include <stdio.h>
2
3  void aplica_desconto(int *preco, int desconto) {
4      if (desconto > *preco) {
5          *preco = 0;
6      } else {
7          *preco = *preco - desconto;
8      }
9  }
10
11 int main() {
12     int precos[3] = {50, 20, 80};
13     for (int i = 0; i < 3; i++) {
14         aplica_desconto(&precos[i], 30);
15         printf("%d\n", precos[i]);
16     }
17     return 0;
18 }
```

- a) Linha 14 — `&precos[i]` está errado; vetores já são endereços, então o correto seria só `precos[i]`.
- b) Linha 4 — `*preco` não pode aparecer numa comparação, apenas do lado esquerdo de uma atribuição.
- c) Linha 3 — uma função `void` não consegue alterar valores, pois não tem `return` para devolvê-los.
- d) Linha 12 — o vetor precisaria ser declarado fora da `main` para poder ser alterado por outra função.
- e) Linha 7 — deveria ser `preco = preco - desconto`, sem `*`, pois `preco` já é o endereço do valor.
- f) O código não tem nenhum erro.

Parte II: Questões Abertas de Programação

Instrução: Desenvolva os programas solicitados utilizando exclusivamente a biblioteca `<stdio.h>`.

Questão 11: Tabuada Validada

Escreva um programa que leia números repetidamente (do-while) até que o usuário digite um valor entre 1 e 10 (inclusive). Cada valor fora do intervalo deve gerar a mensagem de número inválido. Ao receber um valor válido, chame uma função `imprime_tabuada(int n)` que, usando um `for`, imprime a tabuada desse número de 1 a 10.

Entrada exemplo: 0, 12, 3

Saída exemplo:

```
Numero invalido: 0
Numero invalido: 12
3 x 1 = 3
3 x 2 = 6
3 x 3 = 9
...
3 x 10 = 30
```

Questão 12: Pares e Ímpares em Vetor

Leia a quantidade `n` e depois `n` números inteiros, armazenando-os em um vetor. Crie uma função `conta_pares(int v[], int n)` que retorna quantos números pares existem no vetor. Na `main`, use o retorno dessa função para exibir a quantidade de pares e a de ímpares.

Dica: Um número é **par** quando o resto da divisão dele por 2 é zero (`n % 2 == 0`). Se você já sabe quantos são pares, a quantidade de ímpares é o total menos os pares.

Entrada exemplo: `n=6 → 4 7 10 3 8 1`

Saída exemplo:

```
Pares: 3
Impares: 3
```

Questão 13: Potência com While

Escreva uma função `potencia(int base, int expoente)` que calcule base elevado a expoente usando um `while` (sem usar nenhuma função pronta). Leia a base e o expoente na `main` e imprima o resultado.

Dica: Potência: elevar um número a um expoente é multiplicá-lo por ele mesmo tantas vezes quanto o expoente indica. Por exemplo, 2 elevado a 5 é $2 * 2 * 2 * 2 * 2 = 32$. Todo número elevado a 0 é 1 — por isso o resultado pode começar valendo 1.

Entrada exemplo: 2 10

Saída exemplo:

```
Resultado: 1024
```

Questão 14: Caixa Eletrônico

Implemente um caixa eletrônico com do-while + switch, com as opções 1 - Depositar, 2 - Sacar, 3 - Ver saldo e 0 - Sair. O saldo é um int que começa em 0 e fica na main. Crie as funções depositar(int *saldo, int valor) e sacar(int *saldo, int valor), que alteram o saldo original por meio de ponteiro. Se o saque for maior que o saldo, a função sacar deve exibir "Saldo insuficiente" e não alterar nada. O programa volta ao menu até o usuário escolher Sair.

Dica: Como o saldo está na main e precisa ser alterado dentro das funções, ele deve ser passado **por referência**: a função recebe int *saldo, altera com *saldo = ... e a chamada envia o endereço, depositar(&saldo, valor).

Entrada exemplo: 1 → 100, depois 2 → 30, depois 2 → 500, depois 3, depois 0

Saída exemplo:

```
Saldo insuficiente
Saldo: 70
```

Questão 15: Login com Tentativas

Escreva uma função confere_senha(int senha) que retorna 1 se a senha for igual a 4321 e 0 caso contrário. Na main, use um while para dar ao usuário **no máximo 3 tentativas**. A cada erro, informe quantas tentativas restam; se acertar, mostre "Acesso liberado" e pare de pedir a senha; se errar as 3, mostre "Conta bloqueada".

Entrada exemplo: 1111, 2222, 4321

Saída exemplo:

```
Senha incorreta. Restam 2 tentativas
Senha incorreta. Restam 1 tentativas
Acesso liberado
```